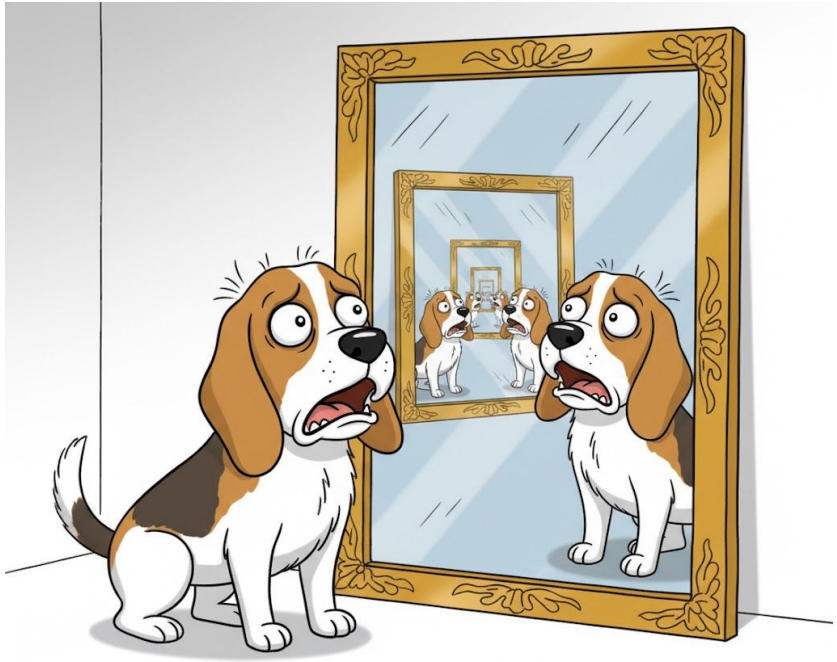
Programação Funcional - Recursão

universidade = "Universidade Federal de Alfenas"
professor = "Romário da Silva Borges"

Na aula de hoje aprenderemos...

- Recursão



Lembrando...

- O paradigma funcional é declarativo e baseado em expressões. Ou seja, você descreve o **que é** o resultado, não **como** chegar até ele.
- Em Haskell, os valores são **imutáveis**.
- Para chamarmos uma função:

  somaValoresLista lista

Contextualizando

- Em linguagens imperativas, usamos Loops para repetir processos. Como podemos realizar isso em Haskell?

- Se não existem variáveis mutáveis, o que aconteceria se tentássemos usar um *loop* **(for, while)** que depende de um contador que muda a cada iteração?

Em Haskell usamos Recursão!

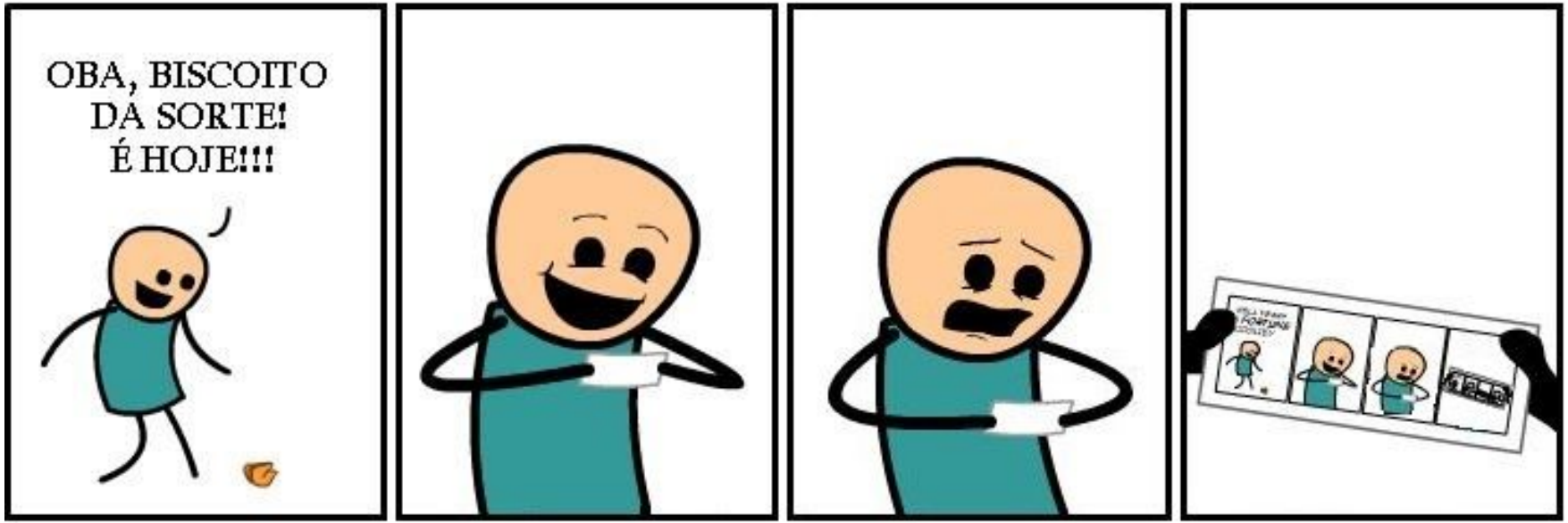
Recursão

Recursividade é o mecanismo de programação no qual uma definição de função ou de outro objeto **refere-se ao próprio objeto sendo definido**. Assim, **função recursiva é uma função que é definida em termos de si mesma**.

Recursividade é o mecanismo básico para repetições nas linguagens funcionais!

Contextualizando

Recursão:



Exemplos

Faça uma função ***soma*** que, dado um valor *n*, soma todos os valores que vai de *n* até 0.

Ex: soma (5)
(5+4+3+2+1+0) = 15

Exemplos

Função *soma* em uma **linguagem imperativa (C)**:

```
int soma(int n) {
  int s = 0;
  for (int i = 1; i <= n; i++) {
    s = s + i;
  }
  return s;
}
```

Exemplos

Função *soma* expressa em **Haskell**:

```
soma :: Int -> Int
soma 0 = 0
soma n = n + soma (n-1)
```

Execução da função *soma* passo a passo

```
soma 5
= 5 + soma 4                                soma n = n + soma (n-1)

= 5 + (4 + soma 3)

= 5 + (4 + (3 + soma 2))

= 5 + (4 + (3 + (2 + soma 1)))

= 5 + (4 + (3 + (2 + (1 + soma 0))))

= 5 + (4 + (3 + (2 + (1 + 0))))
= 15
```

Entendendo um pouco mais...

Função *soma* em **Haskell**:

```
soma :: Int -> Int
soma 0 = 0
soma n = n + soma (n-1)
```

É a nossa condição de parada (ou caso base)!

Imagine o que aconteceria se não tivesse?!

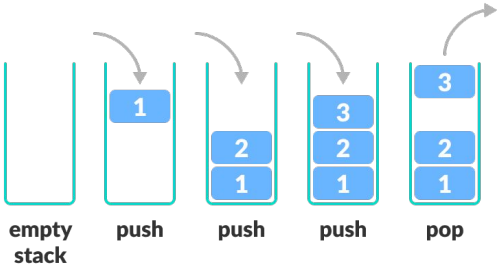
Faça o teste!

Caso Base: Há um ou mais **casos base**, que dizem o que fazer em situações simples, onde não é necessária nenhuma recursão. Assim, não é necessário chamar recursivamente a função, definindo a ordem de parada. **No caso acima, o caso base é 0**.

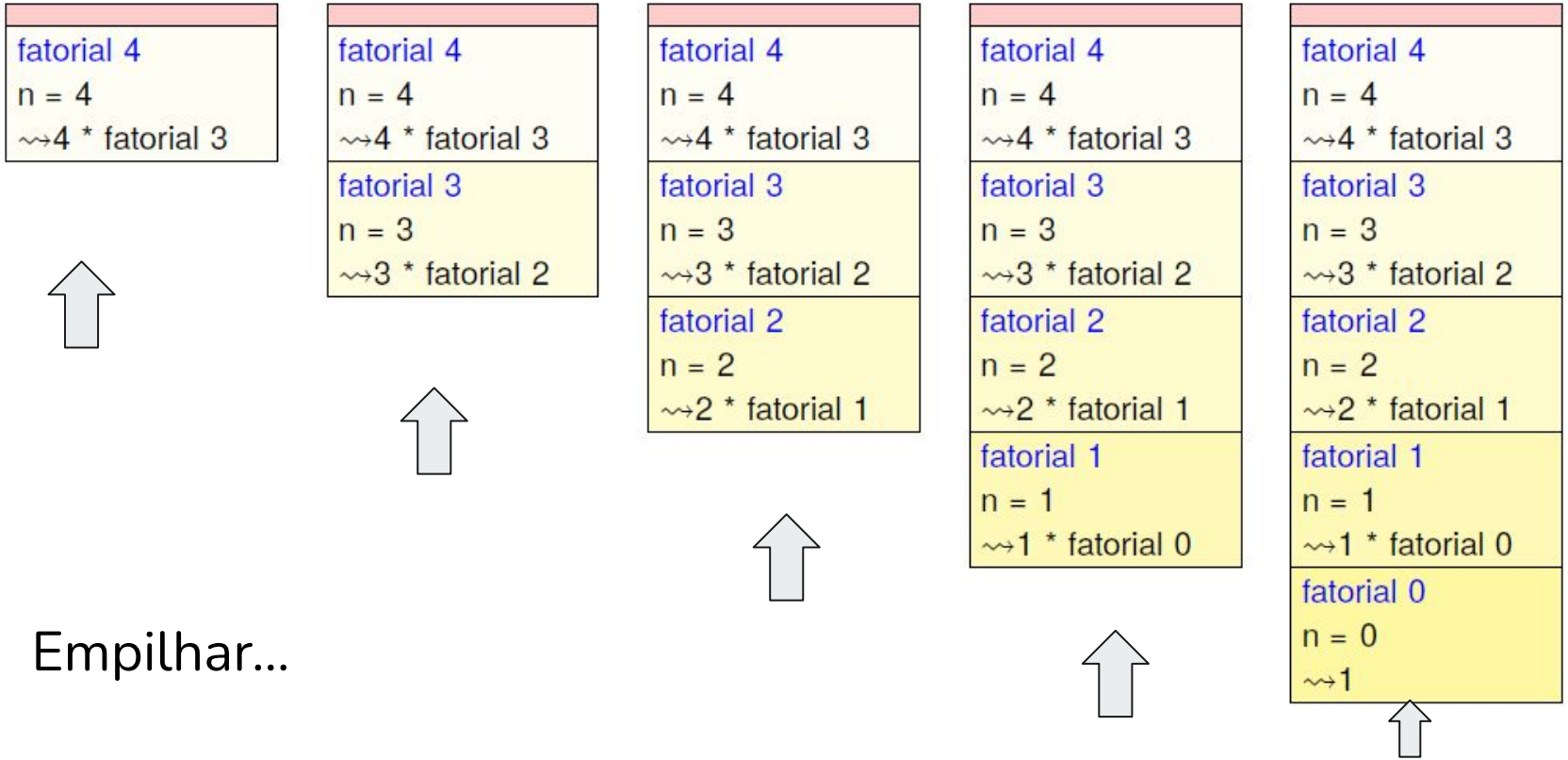
Entendendo um pouco mais...

Em muitas implementações de linguagens de programação, **uma chamada de função usa um espaço de memória, em uma área da memória (pilha)**, onde se armazena: *argumentos da função, variáveis locais, variáveis temporárias, endereço de retorno.*.

Normalmente **para cada nova chamada de função que é realizada, um novo quadro é alocado na pilha**. **Quando a função retorna, o quadro é desalocado**.

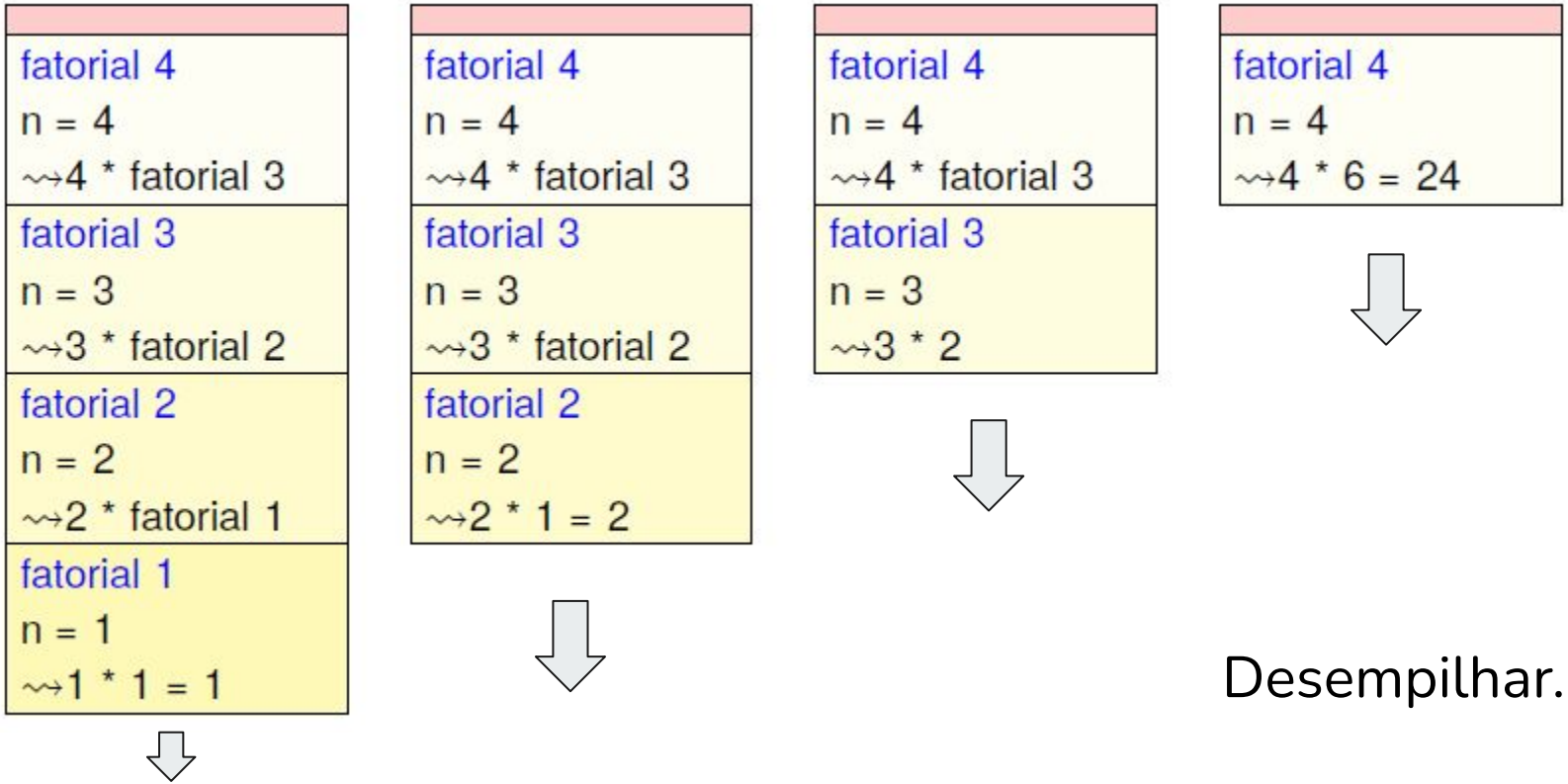


Exemplo da função fatorial



Empilhar...

Exemplo da função fatorial



Desempilhar...

Hora de praticar!

Implemente uma função *somaLista* que soma todos os valores de de uma lista.

Ex:

```
somaLista :: [Int] -> Int
somaLista [5,9,10,10] = 34
```

Hora de praticar! (gabarito)

```
somaLista :: [Int] -> Int
somaLista [] = 0
somaLista (cabeca:cauda) = cabeca + somaLista cauda
```

Hora de praticar!

Implemente uma função *tamanhoLista* que indica o tamanho de uma lista (use recursão para resolver esse problema).

Ex:

```
tamanhoLista :: [Int] -> Int
tamanhoLista [5,9,10,10] = 4
```

Hora de praticar! (gabarito)

```
tamanhoLista :: [Int] -> Int
tamanhoLista [] = 0
tamanhoLista (cabeca:cauda) = 1 + somaLista cauda
```

Considerações finais

Em Haskell, o código é construído com expressões, e a **recursão é a forma natural de repetir cálculos**. já que se trata de uma linguagem que evita estados mutáveis e variáveis que mudam de valor.

curiosidade: A recursão é aceita em linguagens imperativas também!

Referências

- BARROS, J. BERNARDO; ALMEIDA, J. JOÃO. Haskell Tutorial. Departamento de Informática, Universidade do Minho, Braga, Portugal, 1998
- Malaquias, J. R. (2018). *Programação Funcional em Haskell.* Universidade Federal de Ouro Preto, Departamento de Computação.
- DU BOIS, André Rauber. *Programação Funcional com a Linguagem Haskell.* Porto Alegre: Bookman, 2015.

Programação Funcional